

Informe de Algoritmos

Greedy · Huffman · Knapsack 0/1 · LCS

Asignatura:	Análisis y Diseño de Algoritmos
Fecha:	Abril 2026
Lenguaje:	Python 3
Problemas:	4 (Greedy, Huffman, Knapsack, LCS)

Problema 1 — Greedy: Cajero Automático

Se debe dispensar **\$87,500 COP** usando billetes de [50 000, 20 000, 10 000, 5 000, 1 000]. El algoritmo greedy selecciona en cada paso el billete de mayor denominación que no supere el monto restante.

Traza paso a paso

Billete (\$)	Cantidad	Monto restante (\$)
50,000	1	37,500
20,000	1	17,500
10,000	1	7,500
5,000	1	2,500
1,000	2	500

Resultado: [50000, 20000, 10000, 5000, 1000, 1000] → Total: 87,000 COP | Billetes dispensados: 6

¿Qué pasa si se agrega un billete de \$7,000?

Con el billete de \$7,000 incluido, el greedy **sigue siendo óptimo** para este monto particular (87 500). Sin embargo, con denominaciones arbitrarias el greedy puede fallar. Ejemplo clásico: para 30 con [25, 10, 1] el greedy entrega 25+1+1+1+1+1 = 6 monedas, mientras que la solución óptima es 10+10+10 = 3 monedas. Esto se debe a que el greedy no garantiza optimalidad global cuando las denominaciones no forman un sistema canónico.

Configuración	Billetes usados	Cantidad total
Sin \$7 000	[50000, 20000, 10000, 5000, 1000, 1000]	6
Con \$7 000	[50000, 20000, 10000, 7000]	4

Análisis de Complejidad

Sea n = número de denominaciones distintas y k = cantidad máxima de billetes de cada tipo: • Tiempo: $O(n \cdot k)$ en el peor caso. Como k es acotado por el monto ($\text{monto}/\text{denominacion_minima}$), se puede expresar como $O(n \cdot M)$ donde M = monto. • Espacio: $O(n)$ para el arreglo de resultado (o $O(\text{total_billetes})$ si se almacena cada billete). • Para denominaciones canónicas (como COP) el número de billetes es logarítmico respecto al monto, por lo que en práctica es muy eficiente.

Problema 2 — Huffman: Compresión de Logs

Se comprime un log de servidor con las frecuencias: ERROR(45), INFO(120), WARN(30), DEBUG(80), TRACE(15). Huffman asigna códigos de longitud variable: más cortos a los símbolos más frecuentes.

Construcción del árbol (pasos del heap)

Paso	Acción	Heap (frecuencias)
0	Estado inicial	TRACE(15) WARN(30) ERROR(45) DEBUG(80) INFO(120)
1	Fusionar TRACE(15)+WARN(30)	45 ERROR(45) DEBUG(80) INFO(120)
2	Fusionar [45]+ERROR(45)	90 DEBUG(80) INFO(120)
3	Fusionar DEBUG(80)+INFO(120)	200 90
4	Fusionar [90]+[200]	[290] → raíz

Códigos Huffman asignados

Símbolo	Frecuencia	Código	Long. código	Bits totales
INFO	120	0	1	120
DEBUG	80	10	2	160
ERROR	45	110	3	135
WARN	30	1111	4	120
TRACE	15	1110	4	60

Comparación de bits

Métrica	Valor
Bits sin compresión (3 bits/símbolo)	870
Bits con Huffman	595
Ahorro absoluto	275 bits
Porcentaje de ahorro	31.61%

Análisis de Complejidad

- Construcción del árbol: $O(n \log n)$ — n inserciones/extracciones del heap.
- Generación de códigos: $O(n)$ — recorrido DFS del árbol.
- Codificación/decodificación de mensajes: $O(L)$ donde L = longitud del mensaje.
- Espacio: $O(n)$ para el árbol y los códigos.

Problema 3 — Knapsack 0/1: Inversión en Startups

Presupuesto: **\$10 M**. Proyectos disponibles con costo y ROI en millones de USD. Se busca maximizar el ROI total sin exceder el presupuesto (cada proyecto se toma o se deja — no hay fracciones).

Proyectos disponibles

Proyecto	Costo (\$M)	ROI (\$M)	ROI/Costo
HealthTech	3	7	2.33
AI Startup	5	9	1.80
GreenTech	2	4	2.00
Fintech	4	6	1.50

Tabla DP completa — dp[i][w] (ROI máximo)

Item \ Peso	0	1	2	3	4	5	6	7	8	9	10
∅	0	0	0	0	0	0	0	0	0	0	0
HealthTech	0	0	0	7	7	7	7	7	7	7	7
AI Startup	0	0	0	7	7	9	9	9	16	16	16
GreenTech	0	0	4	7	7	11	11	13	16	16	20
Fintech	0	0	4	7	7	11	11	13	16	17	20

Backtracking — cartera óptima

Proyecto	Costo (\$M)	ROI (\$M)	Incluido
HealthTech	3	7	✓
AI Startup	5	9	✓
GreenTech	2	4	✓
Fintech	4	6	✗
TOTAL	10	20	

ROI máximo alcanzable: \$20M con presupuesto de \$10M.

Análisis de Complejidad

- Tiempo: $O(n \cdot W)$ — donde n = número de proyectos y W = capacidad.
- Espacio: $O(n \cdot W)$ — para la tabla dp completa. Se puede reducir a $O(W)$ si no se necesita backtracking.
- Backtracking: $O(n)$ — recorrido lineal de la tabla.
- Para este caso: $n=4, W=10 \rightarrow 40$ celdas calculadas.

Problema 4 — LCS: Control de Versiones

Se comparan dos versiones de un archivo de configuración: **v1:** DEPLOY-PROD-DB-01 | **v2:** DEVELOP-DEBUG-01 La Longest Common Subsequence (LCS) identifica los caracteres comunes en el mismo orden relativo, sin requerir contigüidad.

Tabla DP — $dp[i][j]$ (longitud de LCS)

Columnas = caracteres de v2 | Filas = caracteres de v1

	D	E	V	E	L	O	P	-	D	E	B	U	G	-	0	1	
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
D	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
E	0	1	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
P	0	1	2	2	2	2	2	3	3	3	3	3	3	3	3	3	3
L	0	1	2	2	2	3	3	3	3	3	3	3	3	3	3	3	3
O	0	1	2	2	2	3	4	4	4	4	4	4	4	4	4	4	4
Y	0	1	2	2	2	3	4	4	4	4	4	4	4	4	4	4	4
-	0	1	2	2	2	3	4	4	5	5	5	5	5	5	5	5	5
P	0	1	2	2	2	3	4	5	5	5	5	5	5	5	5	5	5
R	0	1	2	2	2	3	4	5	5	5	5	5	5	5	5	5	5
O	0	1	2	2	2	3	4	5	5	5	5	5	5	5	5	5	5
D	0	1	2	2	2	3	4	5	5	6	6	6	6	6	6	6	6
-	0	1	2	2	2	3	4	5	6	6	6	6	6	6	7	7	7
D	0	1	2	2	2	3	4	5	6	7	7	7	7	7	7	7	7
B	0	1	2	2	2	3	4	5	6	7	7	8	8	8	8	8	8
-	0	1	2	2	2	3	4	5	6	7	7	8	8	8	9	9	9
0	0	1	2	2	2	3	4	5	6	7	7	8	8	8	9	10	10
1	0	1	2	2	2	3	4	5	6	7	7	8	8	8	9	10	11

LCS encontrada: DELOP-DB-01 — Longitud: 11 caracteres

Interpretación en contexto de versionado

La LCS representa la **secuencia de caracteres compartida** entre ambas versiones en el mismo orden. En control de versiones esto equivale a:

- Los caracteres en la LCS son la **base común** que no cambió entre versiones.
- Los caracteres de v1 que *no* están en la LCS fueron **eliminados** o reemplazados.
- Los caracteres de v2 que *no* están en la LCS fueron **insertados**.
- Un diff mínimo (como el de Git) se puede construir a partir de la LCS.

Análisis de Complejidad

Sea $n = |v1| = 17$ y $m = |v2| = 16$:

- Tiempo: $O(n \cdot m)$ — doble bucle para llenar la tabla.
- Espacio: $O(n \cdot m)$ — para la tabla dp completa. Se puede reducir a $O(\min(n,m))$ con optimización de espacio.
- Reconstrucción (backtracking): $O(n + m)$ — recorrido de la tabla.

Conclusiones y Resumen de Complejidades

Algoritmo	Paradigma	Tiempo	Espacio	Óptimo siempre
Greedy (Cajero)	Greedy	$O(n \cdot M)$	$O(n)$	Solo sistemas canónicos
Huffman	Greedy	$O(n \log n)$	$O(n)$	Sí (para freq. dadas)
Knapsack 0/1	Prog. Dinámica	$O(n \cdot W)$	$O(n \cdot W)$	Sí
LCS	Prog. Dinámica	$O(n \cdot m)$	$O(n \cdot m)$	Sí

Los algoritmos **greedy** son muy eficientes pero no siempre garantizan la solución óptima. La **programación dinámica** garantiza optimalidad a costa de mayor uso de memoria. La elección del paradigma depende de las garantías requeridas y los recursos disponibles.